

PPD - Laborator 4 - Documentatie

Rulare	Threads	Readers	Timp (ns)	Raport Ts/Tp
Secvențial	-	-	6 215 340	1
Paralel	4	1	4 908 480	1.26
Paralel	8	1	6 673 640	0.93
Paralel	16	1	14 315 250	0.43
Paralel	4	2	4 452 000	1.39
Paralel	8	2	4 135 020	1.50
Paralel	16	2	5 640 870	1.10

Modul de rulare al aplicației

Pentru măsurarea timpilor de execuție, aplicația paralelă a fost rulată printr-un script PowerShell, folosind următoarea comandă:

```
.\calculateTime.ps1 -CppProgram ".\Tema_4.exe" -Runs 10 -Readers 2
```

Implementare

În implementare am urmărit obiectivele laboratorului, și anume înțelegerea și aplicarea sablonului **producător-consumator**, a mecanismelor de **sincronizare** și a **excluderii mutuale** la nivelul unei structuri de date partajate (lista de rezultate).

Structura de date principală este o listă înlănțuită thread-safe, ale cărei noduri conțin perechi de forma *(ID, Nota)*. Lista garantează că nu există două noduri cu același ID. La citirea unei noi perechi *(id, notaP)*:

- dacă id există deja în listă, notaP se adună la nota acumulată pentru acel student;
- dacă id nu există, se adaugă un nou nod *(ID, NotaP)*.

Metoda A – implementare secvențială

În varianta secvențială, main-thread-ul citește pe rând înregistrările din fișierele `proiect1.txt, ..., proiect10.txt` și actualizează direct lista de rezultate. La final, aceeași execuție scrie conținutul listei în fișierul `rezultate.txt`. În această metodă nu există concurență, deci sincronizarea se reduce la asigurarea corectitudinii operațiilor asupra listei.

Metoda B1 – implementare paralelă cu p thread-uri

În varianta paralelă sunt folosite două tipuri de thread-uri:

- **p_r threads readers:** citesc perechi (*ID, NotaP*) din fișiere și le inserează într-o **coadă nemărginită** (queue). Coadă este implementată manual, fără a folosi structuri de date cu sincronizare integrată.
- **p_w = p – p_r threads workers:** preiau perechi din coadă și actualizează lista înlănțuită *Lista*, aplicând regulile de agregare a notelor.

Comunicarea între threads respectă modelul **producător-consumator**: readers joacă rolul de producători (introduc elemente în coadă), iar workers rolul de consumatori (scot elemente și le inserează în listă). Sincronizarea accesului la coadă și la listă se face:

- la nivel de coadă, prin mecanismul specific limbajului ales (busy-waiting în C++),
- la nivel de listă, prin **blocarea întregii liste** în timpul inserării sau actualizării unui nod, pentru a preveni apariția **data-race-urilor**.

Execuția paralelă se oprește doar după ce toate perechile din toate fișierele au fost procesate și lista conține notele finale. Scrierea fișierului `rezultate.txt` este realizată de main-thread, după ce threads workers au terminat prelucrarea datelor.

Concluzii

1. Impactul paralelizării asupra timpului de execuție

Măsurătorile realizate pentru diferite valori ale numărului de *threads* și pentru rulare secvențială vs. concurentă arată că:

- timpul de execuție scade pe măsură ce crește gradul de paralelism,
- această scădere se manifestă doar până la un anumit prag,

- dincolo de acest prag, creșterea numărului de *threads* nu mai aduce beneficii și poate duce chiar la degradarea performanței, din cauza costurilor de sincronizare și a contenției pe resurse partajate.

2. Analiza raportului Ts/Tp

Evoluția raportului **Ts/Tp** evidențiază faptul că:

- rularea paralelă este superioară celei secvențiale pentru un anumit interval al numărului de *threads*,
- creșterea eficienței nu este liniară cu numărul de *threads*,
- există limitări practice importante: overhead de creare și gestionare a *thread*-urilor, latențe de sincronizare și constrângeri impuse de arhitectura hardware.

3. Concluzie generală

Utilizarea concurenței poate reduce semnificativ timpul de execuție, cu condiția alegerii unui număr de *threads* adecvat caracteristicilor hardware și naturii problemei. Depășirea acestui punct optim duce la câștiguri marginale sau chiar la pierderea beneficiilor, ceea ce subliniază importanța analizării experimentale a aplicațiilor paralele, nu doar a proiectării lor teoretice.

//numarul de elemente din coada sa fie o variabila atomica